음성 AI 전문가 과정 · Day 26 / 40

Day 26

실시간 오디오 기초

Streaming · Jitter · Opus · WebSocket · WebRTC

프레임 · 비트레이트 → 지터 버퍼 · TCP vs UDP → Opus → WebSocket 서버 → WebRTC 해부 → 무엇을 쓸까 · 대역폭
6 주차 M6 · 4 시간 · API 키 없음 · 모든 서버는 Colab 안 127.0.0.1

```
FRAME = 16000 × 20 / 1000 = 320 샘플
PCM16 16 kHz = 256 kbps
Opus 24k → 패킷당 ~60 바이트 (1/10)

재생 지연 60 ms  → 늦은 조각 50%
재생 지연 140 ms → 늦은 조각 1%

WebRTC: sent 160 · lost 0 · jitter 1 ms
```

출력 모양 예시 · 숫자는 시뮬레이션 · 실행마다 다름

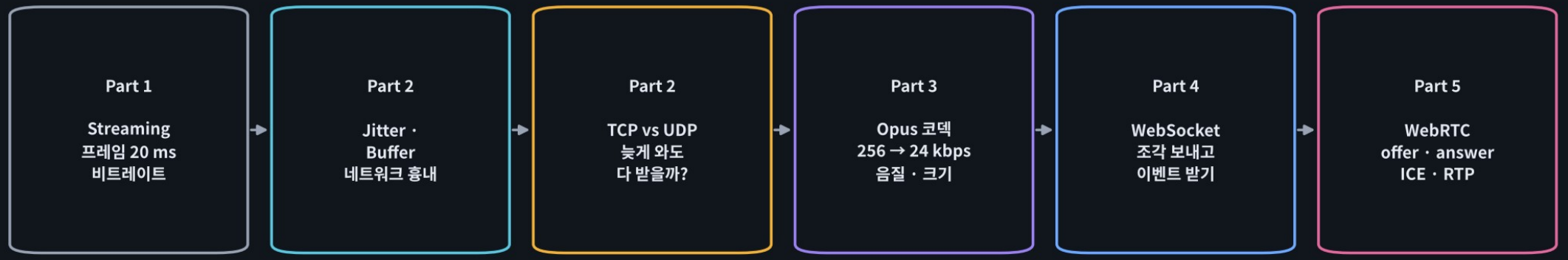
0.0 · 오늘 답할 질문

소리는 어떻게 네트워크를 건너나

1	20 ms 소리 조각은 몇 바이트이고 , 1 초에 몇 개 보내나 ?	Part 1
2	조각이 들쭉날쭉 도착하면 어떻게 고르게 재생하나 ?	Part 2
3	조각을 잃으면 TCP 와 UDP 는 어떻게 다르게 행동하나 ?	Part 2
4	256 kbps 를 24 kbps 로 줄여도 알아들을 수 있나 ?	Part 3
5	WebSocket 으로 조각을 보내고 결과를 받으려면 ?	Part 4
6	WebRTC 는 어떤 단계를 거쳐 소리를 흘려보내나 ?	Part 5

쪼개고 · 흔들고 · 줄이고 · 보내기

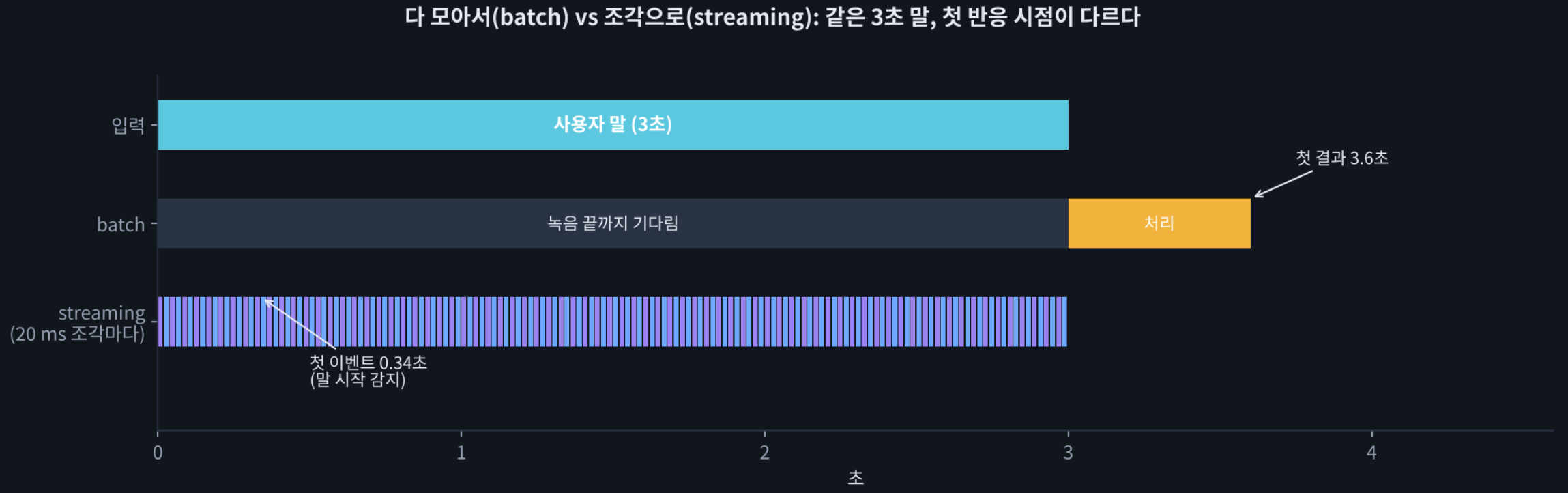
D26 · 소리를 '다 모아서'가 아니라 '조각으로 흘려보내며': 실시간 오디오의 기초



6주차의 바탕: D27 ~ D28 Pipecat (Frame · Pipeline · Transport) · D29 ~ D30 LiveKit (WebRTC Room)이 모두 오늘 개념 위에서 돌아간다

교시	내용	빈칸	애니메이션
1 교시	Part 0 ~ 1 프레임 · 비트레이트 · batch vs streaming	1 · 2	D26-1 · 2 · 3
2 교시	Part 2 지터 · 버퍼 · 적응형 · TCP vs UDP	3 · 4 · 5	D26-4 · 5
3 교시	Part 3 ~ 4 Opus · WebSocket 서버	6 · 7	D26-6 · 7
4 교시	Part 5 ~ 7 WebRTC · 선택 · 대역폭 · 정리	8 · 9	D26-8 · 9

batch(다 모아서) vs streaming(조각으로)



▶ 애니메이션 D26-3 batch vs streaming

🍎 택배 vs 수도관

택배 (batch) 는 다 싸서 한 번에 , 수도관 (streaming) 은 틀면 바로 . 대신 수압 흔들림 (지터) 과 새는 곳 (손실) 이 있다 .

20 ms = 320 샘플 = 640 바이트 · 256 kbps

```
FRAME_MS = 20
FRAME = int(SR * ____ / 1000)      # 1 → 320
BITS = 16
pcm_kbps = SR * ____ / 1000        # 2 → 256
```

형식	kbps	20 ms 바이트
PCM16 48 kHz	768	1,920
PCM16 16 kHz	256	640
μ-law 8 kHz	64	160
Opus 24k	24	60

📐 **핵심 식**

비트레이트 = 샘플링 레이트 × 비트 × 채널

프레임 샘플 = 샘플링 레이트 × ms / 1000

▶ 애니메이션 D26-2 프레임 계산기

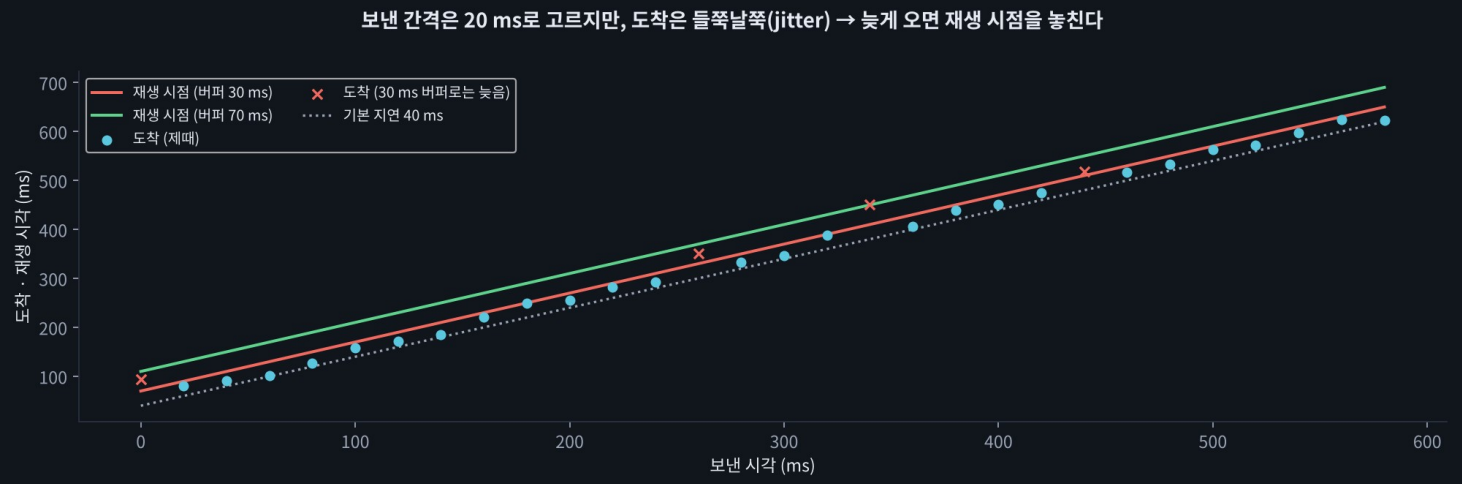
❓ **200 ms 프레임 ? → 머리말 ↓ · 지연 · 손실 영향 ↑**

조각마다 VAD → 말하는 중에 " 말 시작 "

방식	첫 반응 (예)	무엇을 기다리나
streaming (VAD 조각마다)	0.5 ~ 0.7 초	말 시작을 담은 조각 하나
batch (녹음 끝 → 받아쓰기)	녹음 길이 + 처리 ≈ 4 ~ 6 초	말 전체 + 처리 시간

1. 마이크가 20 ~ 32 ms 조각을 만든다
2. 조각이 처리기로 바로 전달된다
3. VAD 가 조각을 판정한다
4. " 말 시작 " 이벤트 → 다음 단계 (STT · 화면 표시) 를 바로 시작

고르게 보내도 들쭉날쭉 도착한다



용어	뜻
지연	보낸 뒤 도착까지
지터	지연의 흔들림
손실	아예 안 옴 (UDP)
재생 시점	보낸 시각 + 재생 지연
PLC	빈 조각 메우기

▶ 애니메이션 D26-4 지터 버퍼 시뮬레이터

📐 **재생 시점 = 보낸 시각 + 재생 지연 · 늦음 = 도착 > 재생 시점**

deadline = send + ____ · late = arrival > ____

```
def playout(send, arrival, delay_ms):
    deadline = send + ____          # 3
    late = arrival > ____           # 4
    return deadline, late
```

재생 지연	늦은 조각 (예, 지터 25 ms)
50 ms	80 % 이상
80 ms	약 8 %
100 ms	약 3 %
140 ms	1 % 이하

🎧 **들어 보기 : 60 ms vs 140 ms**
뚝뚝 끊기는 소리 vs 조금 늦지만 매끈한 소리. 통화 입 - 귀 지연은 약 150 ms 부터 어색해진다 (ITU-T G.114 권고).

지난 지연의 95 번째 백분위로

```
obs = (arrival - send)[:50]            #  관찰한 도착 지연
ADAPT_MS = np.percentile(obs, ____)     # 5
```

방식	재생 지연	늦은 조각 (예)
고정 60 ms	60 ms	50 % 안팎
적응형 p95	약 80 ms	10 % 안팎

💬 **쉽게 말하면** 네트워크가 좋으면 버퍼를 줄이고 , 나빠지면 늘린다 · 늦은 조각 비율과 지연을 맞바꾼다

❓ **처음 50 개만 보면 ? → 나중에 나빠지면 못 따라감 → 계속 갱신해야**

잃었을 때 : 다시 보내며 기다릴까 , 건너뛸까

조각 하나를 잃었을 때: TCP(WebSocket)는 다시 보낼 때까지 뒤 조각도 기다리고, UDP(WebRTC)는 건너뛴다



실시간 음성에서 '늦게 온 조각'은 '안 온 조각'과 같다 → 음성 미디어는 UDP 위 RTP, 제어 · 글자는 TCP

손실 3 % · 재생 100 ms (예)	못 쓴 조각	최대 지연
UDP (WebRTC)	약 10 %	약 150 ms
TCP (WebSocket)	30 % 이상	300 ms 이상

▶ 애니메이션 D26-5 TCP vs UDP

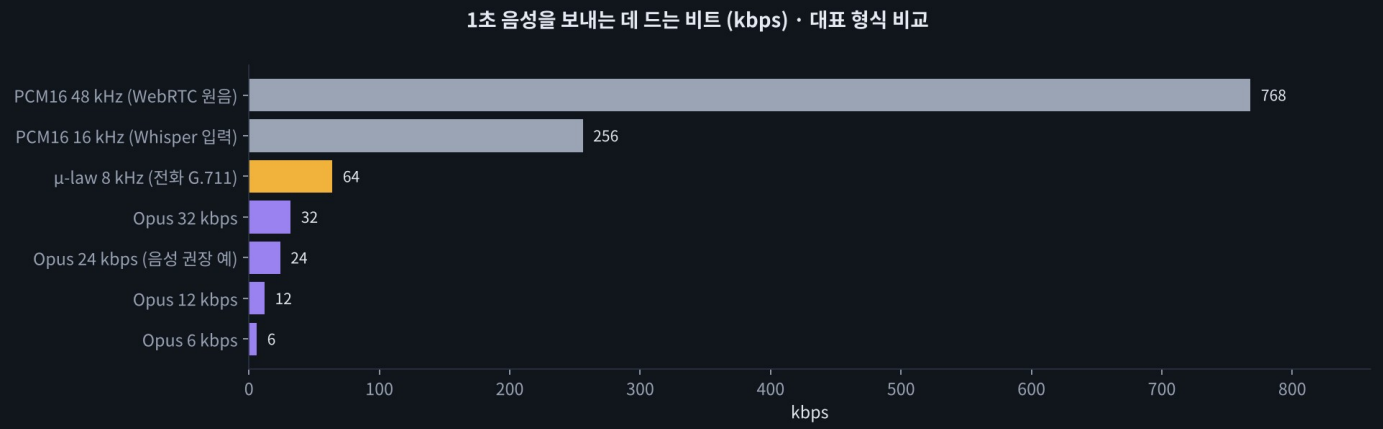
안정된 구간에서는 TCP 도 충분하다

구간	손실 · 지터	흔한 선택
브라우저 · 앱 ↔ 서버 (와이파이 · LTE)	많음 · 들쭉날쭉	WebRTC (UDP · 지터 버퍼 · PLC)
서버 ↔ 서버 (데이터센터 안)	거의 없음	WebSocket (TCP) · 구현 쉬움
전화망 게이트웨이 ↔ 음성 서버	적음	WebSocket (μ-law 8 kHz) 또는 RTP

🍎 **성실한 우체국**

TCP 는 잃어버린 편지를 다시 보낼 때까지 뒤 편지도 붙잡아 두는 성실한 우체국 . 시골길 (손실 많은 구간) 에서는 너무 성실하다 .

256 kbps → 24 kbps, 그래도 알아듣는다



특징	값
내부 샘플링	48 kHz
프레임	2.5 ~ 60 ms (기본 20)
비트레이트	6 ~ 510 kbps
음성 설정	application=voip
선택 기능	FEC · DTX

▶ 애니메이션 D26-6 비트레이트와 크기

enc.bit_rate = ____ · 크기 · mel 거리 · CER

```
enc = av.CodecContext.create("libopus", "w")
enc.sample_rate, enc.layout = 48000, "mono"
enc.bit_rate = ____                # 6
enc.options = {"application": "voip",
               "frame_duration": "20"}
```

코덱	실제 kbps (예)	받아쓰기 경향
Opus 6k	약 5	CER 상승
Opus 12k	약 9	조금 흐림
Opus 24k	약 17	원음과 비슷
μ-law 8 kHz	64	ㅅ ㅆ ㅈ 약함

음성 비서는 "STT 가 듣는 것 " 기준 → 16 ~ 32 kbps 면 무난 · 전화 연동은 8 kHz 조건으로 따로 평가

바이너리는 소리 , 텍스트 (JSON) 는 신호

방향	메시지	예
클 → 서	바이너리	20 ms 조각 (640 바이트)
클 → 서	텍스트	{"codec": "opus"} · "end"
서 → 클	텍스트	{"event": "speech_start"}

1. 서버 띄우기 (127.0.0.1)
2. 연결 + 설정 JSON
3. 20 ms 마다 조각 보내기
4. 조각마다 이벤트 받기
5. "end" → 요약

▶ 애니메이션 D26-7 WebSocket 메시지 흐름

🍎 무전기

한 번 채널을 맞춰 두면 양쪽이 아무 때나 말한다 . 소리는 바이트로 , 신호는 JSON 글자로 .

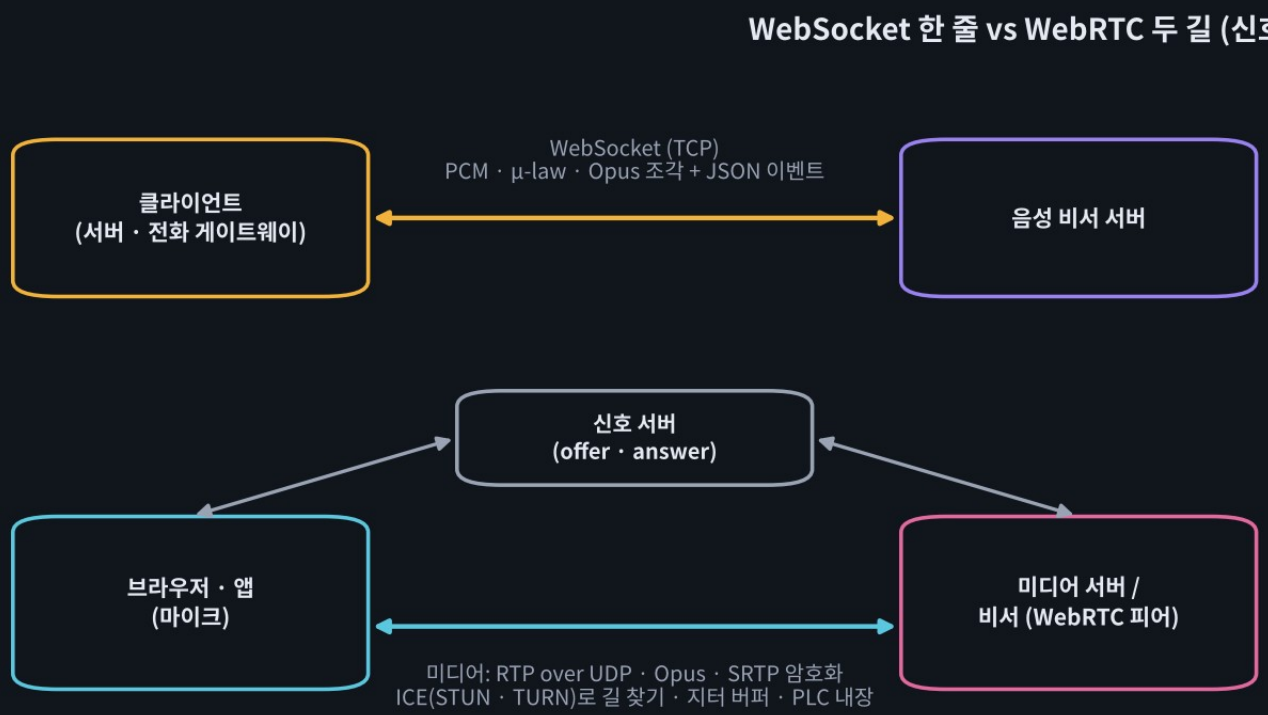
await ws.send(chunk.____())

```
async with connect(f"ws://127.0.0.1:{WS_PORT}") as ws:
    await ws.send(json.dumps({"codec": "pcm"}))
    for ch in chunks:
        await ws.send(ch.____())          # 7
        ev = json.loads(await ws.recv())
        await asyncio.sleep(...)   # 20 ms 간격
```

결과 (예)	값
조각	160
왕복 p50	1 ~ 2 ms
말 시작 감지	0.5 초
PCM 바이트	102,400
Opus 바이트	약 7,000

실제 서비스 : + 네트워크 왕복 · TLS(wss://) · 서버 처리 · TCP 재전송 튐

신호 (offer · answer) 와 미디어 (RTP) 는 다른 길



	WebSocket	WebRTC
전송	TCP (순서 · 빠짐 없음)	UDP + RTP (늦으면 버림)
코덱	직접 정함 (PCM · Opus …)	Opus 협상 내장
NAT · 방화벽	보통 문제 없음 (443)	ICE · STUN · TURN 필요
지터 버퍼 · PLC	직접 구현	내장
잘 맞는 곳	서버 ↔ 서버 · 전화 연동	브라우저 · 앱 마이크

▶ 애니메이션 D26-8 WebRTC 연결 단계

pc2.setRemoteDescription(pc1.____)

단계	무엇을
① offer	pc1: "Opus 로 보낼 수 있어 "
② 신호 전달	offer → pc2 (아무 방법)
③ answer	pc2: " 좋아 , 받을게 "
④ ICE	닿는 주소 · 포트 찾기
⑤ DTLS · SRTP	키 교환 · 암호화
⑥ RTP	20 ms Opus 패킷 흐름

```
m=audio … UDP/TLS/RTP/SAVPF 96
a=rtpmap:96 opus/48000/2
a=candidate:… 127.0.0.1 … typ host
a=fingerprint:sha-256 …
a=setup:actpass
```

결과 (예): sent 160 · lost 0 · 지터 1 ms

신호 서버 · STUN · TURN · HTTPS

필요한 것	왜	없으면
신호 서버	offer · answer · 후보 전달	연결을 시작할 수 없음
STUN	NAT 바깥에서 보이는 내 주소 알기	집 · 회사 NAT 뒤에서 길을 못 찾음
TURN	직접 길이 막히면 중계	UDP 막힌 회사망에서 실패 (중계는 대역폭 비용)
HTTPS	브라우저 마이크 권한은 안전한 출처에서만	마이크를 쓸 수 없음

🍎 **국제 화상 회의 잡기**

비서끼리 이메일 (신호) 로 시간 · 링크를 정하고 , 방화벽을 통과할 길 (ICE · TURN) 을 찾은 뒤에야 대화 (RTP) 가 시작된다 .

사람 쪽은 WebRTC, 서버끼리는 WebSocket

상황	추천
브라우저 · 앱 마이크	WebRTC
전화 게이트웨이 ↔ 서버	WebSocket (μ-law)
서버 ↔ 클라우드 STT · TTS	WebSocket
여러 사람이 한 방	WebRTC + SFU

▶ 애니메이션 D26-9 무엇을 쓸까

```
overhead = 40 B × 8 × 50 /s = 16 kbps
call = (codec + overhead) * ____   # 9
```

100 통화	합계
PCM16 16 kHz	54.4 Mbps
μ-law 8 kHz	16.0 Mbps
Opus 24k	8.0 Mbps

조각으로 흘려보내고 , 늦으면 버리고 , 작게 보낸다

개념	코드
프레임	`FRAME = SR * FRAME_MS / 1000`
지터 버퍼	`deadline = send + delay_ms`
적응형	`np.percentile(obs, 95)`
Opus	`enc.bit_rate = kbps * 1000`
WebSocket	`ws.send(chunk.tobytes())`
WebRTC	`setRemoteDescription(pc1.localDescription)`
대역폭	`(codec + overhead) * 2`

다음 : D27 Pipecat 기초

오늘 손으로 만든 조각 · 흐름 · 통로를 Pipecat 이 Frame · Pipeline · Processor · Transport 로 묶습니다 . VAD → STT → LLM → TTS 를 흘려보내는 비서 .

`worksheet.html` 40 문항

개념 워크시트 · 노션 제출

`animation.html` 9 개 탭

인터랙티브 애니메이션

`D26_ 실시간오디오 _ 실습 .ipynb` 빈칸 9

실습 노트북 · 키 없음

온라인 자료 imguru-mooc.github.io/AI_Voice/day26